

University of Chittagong

Department of Computer Science & Engineering

CSE 712: Compiler Lab

Final Project: Design and Implementation of a MiniLang Compiler

1 Project Description:

Design and implement a complete compiler for a small statically-typed programming language named **MiniLang**. MiniLang is a simplified C-like language. Your compiler must process a MiniLang source program and translate it into **Three-Address Intermediate Code (TAC)** while performing full lexical, syntactic, and semantic analysis.

Reference specification: <https://minilang.readthedocs.io/en/latest/>

2 Source Language: MiniLang Specification:

2.1 Supported Constructs:

- Variable declarations (`int`, `bool`)
- Assignment statements
- Arithmetic expressions (`+`, `-`, `*`, `/`)
- Relational expressions (`<`, `>`, `==`, `!=`)
- `if--else` conditionals
- `while` loops
- `print()` statement
- Block scoping using `{ }`

2.2 Not Supported:

- Arrays
- Functions
- Pointers
- Floating-point values

3 Input and Output:

- **Input:** `.ml` file containing MiniLang program
- **Output:** Three-Address Code representation printed to file `output.tac`

4 You MUST implement all phases listed below:

4.1 Lexical Analysis (Flex):

Create a scanner that:

- Recognizes keywords: `int`, `bool`, `if`, `else`, `while`, `print`
- Identifies:
 - Identifiers
 - Integer constants
 - Operators
 - Delimiters
- Ignores whitespace and comments
- Reports lexical errors with line numbers

4.2 Syntax Analysis (Bison)

Implement grammar rules to validate program structure. The parser must:

- Resolve ambiguity
- Enforces precedence and associativity
- Produces an Abstract Syntax Tree (AST)
- Reports meaningful syntax errors

4.3 Semantic Analysis:

Implement static checks using a symbol table with scope management:

- Undeclared variables
- Multiple declarations in same scope
- Type mismatch in expressions
- Invalid conditional expressions
- Detection of semantic errors

4.4 Intermediate Code Generation

Translate the AST into Three-Address Code (TAC):

- Generate temporaries and labels
- Represent control flow explicitly
- Produce a machine-independent IR in `output.tac`

4.5 Code Optimization

Apply basic intermediate-level optimizations such as:

- Constant folding
- Dead code elimination (where applicable)
- Redundant temporary removal
- Simple algebraic simplifications

Optimizations should operate on TAC without altering program semantics.

4.6 Target Code Generation

Generate a simplified target representation (pseudo-assembly or stack-machine code) from optimized TAC:

- Map TAC instructions to low-level operations
- Handle variable storage and temporaries
- Emit labeled control-flow instructions

4.7 Error Handling:

Your compiler must handle:

- Lexical errors
- Syntax errors
- Semantic errors

5 Required Project Structure:

```
MiniCompiler/  
  lexer.l  
  parser.y  
  ast.c / ast.h  
  symbol_table.c  
  semantic.c  
  codegen.c  
  main.c  
  Makefile  
  testcases/
```

6 Compilation Instructions:

Compilation must succeed using:

```
flex lexer.l  
bison -d parser.y  
gcc lex.yy.c parser.tab.c *.c -o minicompiler
```

Provide a Makefile automating this process.

7 Complexity Evaluation of Your Implementation

In addition to implementing the compiler, you must perform a formal complexity analysis of **your actual solution code** for each compilation phase. This analysis should be based on both theoretical reasoning and empirical observation.

8 Written Report Requirements:

Submit a concise technical report (PDF) that clearly documents the design, implementation, and analysis of your compiler. The report must include the following sections (but are not limited to):

1. Language Design Decisions

- Overview of MiniLang features implemented

- Rationale for simplifying assumptions
2. **Lexical Analysis**
 - Regular expressions and token definitions
 - Handling of whitespace, comments, and errors
 - **Complexity Analysis:** Time and space complexity of scanning
 3. **Syntax Analysis**
 - Grammar design and precedence rules
 - Conflict resolution strategy (if any)
 - **Complexity Analysis:** Cost of parsing
 4. **Abstract Syntax Tree (AST)**
 - AST structure and node types
 - Why AST is used instead of parse tree
 - **Complexity Analysis:** Memory and construction cost
 5. **Symbol Table and Semantic Analysis**
 - Scope management approach
 - Type-checking implementation
 - Error detection strategy
 - **Complexity Analysis:** Lookup, insertion, and traversal cost
 6. **Intermediate Code Generation**
 - TAC design and instruction format
 - Translation of conditionals and loops
 - **Complexity Analysis:** Cost of AST traversal and code emission
 7. **Overall Compiler Complexity**
 - Combined time complexity of all phases
 - Space usage of AST, symbol table, and TAC
 - Discussion of whether the compiler operates in linear time
 8. **Sample Compilation Results**
 - Example MiniLang program
 - Generated Three-Address Code output